Lab Report

Computer Architecture Laboratory

Assignment 6

Anshuman Rahul
CS24BT060

1 Configuration

- Main Memory Latency: 40 cycles
- ALU Latency: 1 cycle
- Multiplier Latency: 4 cycles
- Divider Latency: 10 cycles

2 Throughput with cache

2.1 L1d-Cache fixed: 1kB, 4 cycle latency

Program	**IPC**			
	16B	**128B**	**512B**	**1kB**
even-odd.asm	0.0231	0.0226	0.0222	0.0218
descending.asm	0.0162	0.1047	0.0936	0.0846
fibonacci.asm	0.0172	0.0473	0.0447	0.0424
palindrome.asm	0.0210	0.0597	0.0571	0.0547
prime.asm	0.0212	0.0463	0.0441	0.0422

2.2 L1i-Cache fixed: 1kB, 4 cycle latency

Program	**IPC**			
	16B	**128B**	**512B**	**1kB**
even-odd.asm	0.0221	0.0220	0.0219	0.0218
descending.asm	0.0722	0.0900	0.0873	0.0846
fibonacci.asm	0.0433	0.0430	0.0427	0.0424
palindrome.asm	0.0549	0.0549	0.0548	0.0547
prime.asm	0.0424	0.0423	0.0423	0.0422

3 Comparison of throughput without cache

Program	IPC without cache	Configuration		IPC with cache	Performance gain
		L1d	L1i		
even-odd.asm	0.0239	1024B	16B	0.0231	-3.35%
descending.asm	0.0185	1024B	128B	0.1047	465.95%
fibonacci.asm	0.0204	1024B	128B	0.0473	131.86%
palindrome.asm	0.0215	1024B	128B	0.0597	177.67%
prime.asm	0.0211	1024B	128B	0.0463	119.43%

There is an overall loss of performance for the even-odd.asm program. This can be attributed to the small size and short execution time of the program, which results in limited temporal and spatial locality. Consequently, the caches experience mostly compulsory misses, with insufficient reuse to offset the added latency of cache accesses. As a result, the overhead of introducing caches outweighs any performance gains, leading to a net decrease in performance.

L1d = 1024B and L1i=128B seems to be the best configuration for most of the programs in this implementation of the simulator.

4 Graphs and Observations

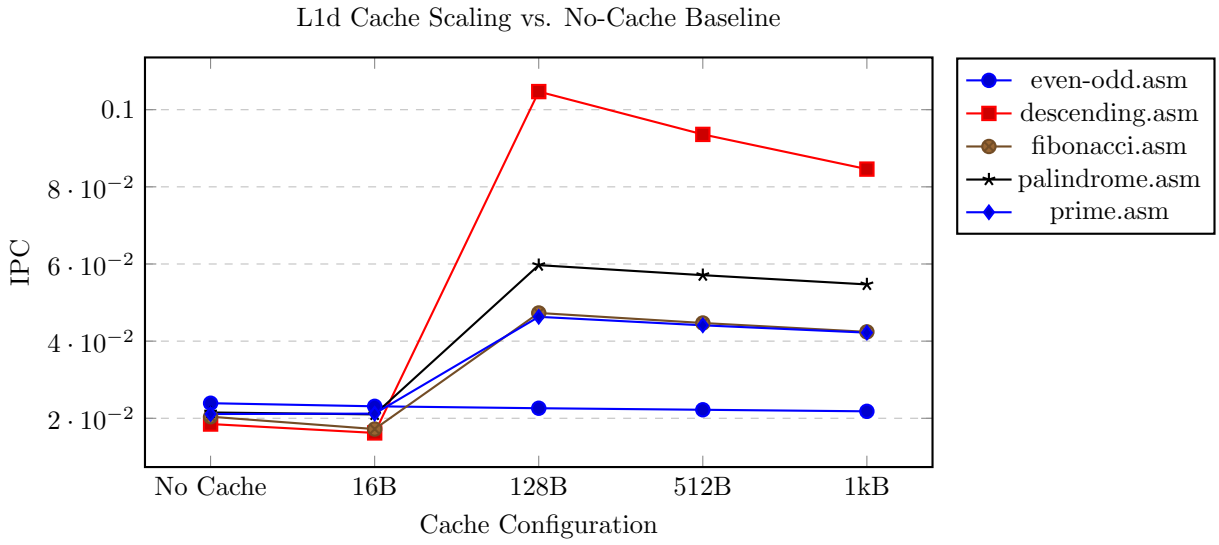

Figure 1: Impact of L1-Data cache size on IPC compared to the non-cached baseline.

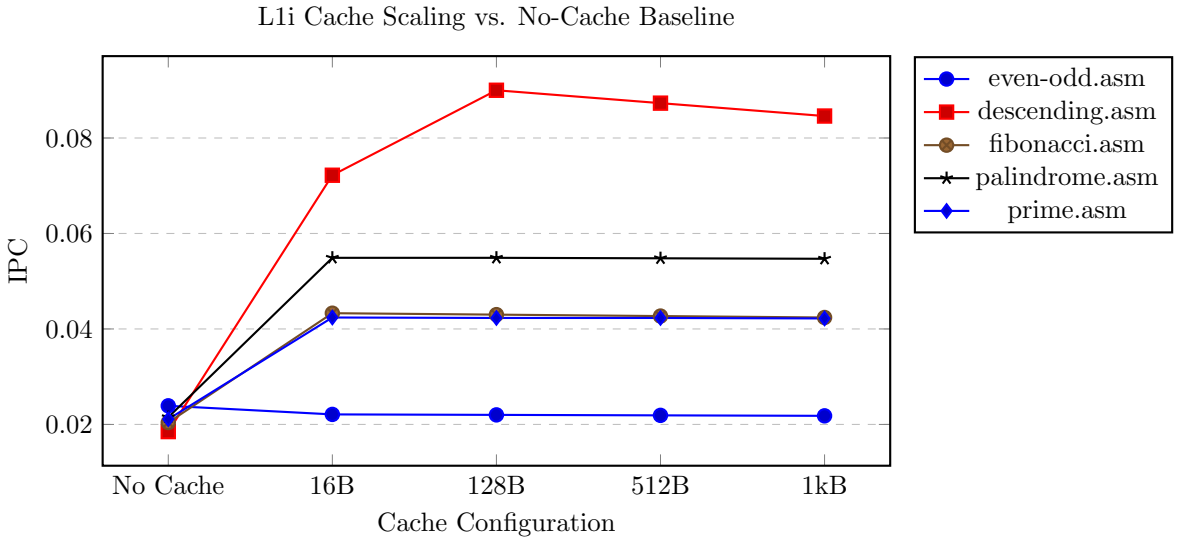

Figure 2: Instruction cache performance scaling relative to the original non-cached processor.

With even-odd.asm being the exception, all programs showed increase in performance after the introduction of a cache.

In the graph of L1d cache size vs IPC, it can be observed increasing the cache size doesn't necessarily lead to increase in performance as we also need to take into account the latency of the cache as well which increases as the size increases.

In the graph of L1i cache vs IPC, it can be observed the IPC mostly plateaus implying fetching instructions isn't necessarily the main bottleneck for the given set of programs.

The slight decline in IPC at larger cache sizes (512B and 1kB) suggests that the working sets of these specific programs already fit within 128B. Beyond this point, the increased cache access latency provides no further benefit in terms of hit rate, but increases the cost of every access, thereby reducing overall throughput

5 Toy Benchmarks to demonstrate performance gains

Increase in performance after increasing L1i-cache from 16B to 128B

```
.text
main:
    addi %x0, 10000, %x3
    addi %x0, 1, %x4
loop:
    beq %x3, %x0, end
    add %x5, %x4, %x5
    add %x6, %x4, %x6
    add %x7, %x4, %x7
    add %x8, %x4, %x8
    add %x9, %x4, %x9
    add %x10, %x4, %x10
    sub %x3, %x4, %x3
    jmp loop
end:
    end
```

Listing 1: Simple RISC L1i Cache Benchmark Program

IPC for 16B = 0.0220
IPC for 128B = 0.4489
%age increase = 1940.45%

Increase in performance after increasing L1d-cache from 16B to 128B

```
.data
a:
    1
    2
    3
    4
    5
.text
main:
    addi %x0, 1000, %x3
    addi %x0, 1, %x4
loop:
    beq %x3, %x0, end
    sub %x5, %x5, %x5
    load %x5, $a, %x6
    add %x4, %x5, %x5
```

```
    load %x5, $a, %x7
    add %x4, %x5, %x5
    load %x5, $a, %x8
    add %x4, %x5, %x5
    load %x5, $a, %x9
    add %x4, %x5, %x5
    load %x5, $a, %x10
    add %x4, %x5, %x5
    sub %x3, %x4, %x3
    jmp loop
end:
    end
```

Listing 2: Simple RISC L1d Cache Benchmark Program

IPC for 16B = 0.0224
IPC for 128B = 0.2514
%age increase = 1022.32